

Guide de l'application

Examens blancs LFJP

Présentation fonctionnelle et technique

Document généré le 20 août 2026

Portail de consultation des plannings, salles, élèves, surveillances et convocations.

1. Présentation générale

L'application Examens blancs LFJP est un portail web destiné à centraliser l'organisation des examens blancs et de plusieurs épreuves de l'établissement. Elle permet aux enseignants et responsables d'accéder rapidement aux informations utiles depuis une seule interface.

- Consulter les dates et les plannings d'examens.
- Visualiser les salles, les élèves et les affectations.
- Organiser les surveillances BAC et DNB.
- Générer des convocations et des documents PDF.
- Exporter certains plannings au format CSV.

Nature de l'application

L'application fonctionne entièrement côté navigateur. Les données sont actuellement stockées dans des fichiers TypeScript intégrés au projet. Il n'y a pas de serveur applicatif, d'authentification ou de base de données connectée.

Technologies principales

- React 18 et TypeScript pour l'interface et la logique.
- Vite pour le développement et la compilation de production.
- React Router pour la navigation entre les pages.
- Tailwind CSS pour les styles.
- Lucide React pour les icônes.
- jsPDF et html2canvas pour les exports et impressions.

2. Parcours utilisateur

Accueil

L'utilisateur arrive sur une page d'accueil présentant l'espace examens et une série de cartes classées par date. Chaque carte ouvre la page correspondant à une épreuve ou à un planning.

```
/
/examens-blancs
/examens-blancs/mathematiques-2026-02-13
/examens-blancs/oraux-eaf-2026-05
/surveillances-bac-dnb
```

Consultation d'un examen

Les pages d'examen affichent un en-tête avec le titre, la date et les indicateurs clés, puis des vues adaptées au besoin : organisation complète, planning par enseignant ou jury, planning par salle, élèves et convocations.

Navigation

Les tableaux de bord utilisent des onglets accessibles au clavier. Les flèches gauche et droite changent d'onglet, tandis que les touches Home et End permettent d'aller au premier ou au dernier onglet.

3. Modules fonctionnels

Tableau de bord des examens blancs

- Configuration des salles et capacités.
- Liste des élèves de Première et Terminale.
- Affectation des enseignants aux missions de surveillance.
- État des salles et informations pratiques.
- Génération de convocations.

Examens de mathématiques et DNB

Les pages de mathématiques réutilisent une même structure. Seules les données changent selon l'examen : date, élèves, salles, enseignants, épreuves et horaires.

Oraux EAF et Grand Oral

Ces modules sont construits autour de listes de candidats. Les candidats peuvent être regroupés par date, jury ou salle. Les tableaux affichent les horaires de convocation, les horaires de passage, les classes et les salles.

Surveillances BAC et DNB

Le module de surveillance permet de basculer entre les plannings BAC et DNB, de rechercher un surveillant et de produire une convocation individuelle. L'heure de présence est calculée automatiquement 30 minutes avant le début de l'épreuve.

4. Architecture technique

Point d'entrée et routes

Le fichier `src/main.tsx` monte React et BrowserRouter. Le fichier `src/app/App.tsx` déclare les routes et charge les pages avec `lazy()` afin de réduire le chargement initial.

Organisation par fonctionnalités

Le code est organisé par domaine dans `src/features`. Chaque fonctionnalité regroupe ses pages, composants, données, hooks, contextes, services et utilitaires. Les composants génériques sont placés dans `src/shared`.

Gestion d'état

L'état d'interface est local aux composants React. Les tableaux de bord utilisent des Contexts pour partager l'onglet actif, le conteneur de rendu et le jeu de données de l'examen. Les données ne sont pas persistées dans `localStorage`.

Données

Les données de l'établissement sont déclarées comme constantes TypeScript : annuaire des enseignants, élèves, salles, horaires, jurys et missions. Pour modifier une organisation, il faut donc modifier le code source puis reconstruire l'application.

5. Documents et exports

PDF

Les exports PDF sont générés directement dans le navigateur avec jsPDF. Les tableaux sont dessinés ligne par ligne, avec gestion des retours à la ligne et des pages multiples.

CSV

Les pages d'oraux créent un fichier CSV avec séparateur point-virgule, encodage UTF-8 et marqueur BOM afin de faciliter l'ouverture dans Excel.

Impression

Le module de surveillance peut ouvrir une fenêtre d'impression dédiée. html2canvas est également utilisé pour transformer certains éléments HTML en image avant création du PDF.

Limites des documents

- Les documents reflètent les données intégrées au moment de la compilation.
- Le cachet et certains logos utilisent des ressources externes.
- Les bloqueurs de fenêtres contextuelles peuvent empêcher l'impression.
- Les exports ne constituent pas une sauvegarde centralisée.

6. Installation et validation

```
npm install --legacy-peer-deps
npm run dev
npm run build
```

La commande npm install standard rencontre actuellement un conflit entre ESLint 9 et @typescript-eslint 7. L'installation avec --legacy-peer-deps permet de lancer le projet.

Le build de production a été vérifié avec succès. ESLint ne s'exécute pas avec la configuration actuelle, car ESLint 9 attend un fichier eslint.config.js alors que le projet possède encore une configuration .eslintrc.cjs.

Publication

Le dépôt GitHub est : https://github.com/informatique-cmd/examens_blanc_lfjp. La branche locale main est synchronisée avec origin/main.

7. Points d'amélioration

- Ajouter une base de données ou un CMS pour modifier les plannings sans changer le code.
- Ajouter une authentification et des droits d'accès, car l'application contient des données nominatives.
- Ajouter des tests unitaires et des tests de parcours navigateur.
- Moderniser la configuration ESLint.
- Résoudre le conflit de versions ESLint et réduire les vulnérabilités npm.
- Héberger localement les logos, le cachet et les autres ressources nécessaires.
- Prévoir une gestion des erreurs pour les exports et les liens externes.
- Optimiser le poids des gros bundles, en particulier jsPDF.

Conclusion

L'application est déjà un portail opérationnel de consultation et de génération de documents pour les examens. Son architecture par fonctionnalités et ses jeux de données typés facilitent l'ajout de nouvelles sessions. Le principal changement de niveau serait de remplacer les données statiques par une source administrable et sécurisée.